



File I/O

Mahir Labib Dihan
Lecturer, CSE, BUET



Table of Contents

1. Console I/O
2. Formatted I/O
3. File I/O
4. Summary

Console I/O



Standard I/O Streams

C provides three standard streams:



- **stdin:** Standard input (keyboard by default)
- **stdout:** Standard output (screen by default)
- **stderr:** Standard error (for error messages)

Character I/O: getchar() and putchar()

```
#include <stdio.h>

int main() {
    char c;

    printf("Enter a character: ");
    c = getchar(); // Read one character from stdin

    printf("You entered: ");
    putchar(c); // Write one character to stdout
    putchar('\n');

    return 0;
}
```

Example run:

Enter a character: A

You entered: A

String I/O: fgets() and puts()

```
#include <stdio.h>

int main () {
    char name[50];

    printf("Enter your name: ");
    fgets(name, 50, stdin); // Safe: specify max length

    printf("Hello, ");
    puts(name); // Automatically adds newline

    return 0;
}
```

Warning

Never use `gets()` ! It's unsafe (no buffer overflow protection).
Always use `fgets(buffer, size, stdin)` instead.

Formatted I/O



printf(): Format Specifiers

Specifier	Type	Example
%d or %i	int	printf("%d", 42);
%u	unsigned int	printf("%u", 42);
%ld	long int	printf("%ld", 123456789L);
%f	float/double	printf("%f", 3.14);
%.2f	2 decimal places	printf("%.2f", 3.14159);
%e	scientific notation	printf("%e", 12345.0);
%c	char	printf("%c", 'A');
%s	string	printf("%s", "Hello");
%p	pointer address	printf("%p", &x);
%x	hexadecimal	printf("%x", 255);
%%	literal %	printf("%%");

printf(): Width and Precision

```
int x = 42;
double pi = 3.14159265;

// Width: minimum characters to print
printf("[%5d]\n", x);      // [  42] (right-aligned)
printf("[%5d]\n", x);      // [42   ] (left-aligned)
printf(" [%05d]\n", x);    // [00042] (zero-padded)

// Precision: decimal places for floats
printf("[%f]\n", pi);      // [3.141593]
printf(" [%.2f]\n", pi);   // [3.14]
printf(" [%8.2f]\n", pi);  // [      3.14]

// For strings: maximum characters
printf("[%5.5 s]\n", "Hello World"); // [Hello]
```

scanf(): Reading Input

```
int age;  
float height;  
char name[50];  
char grade;  
  
// Reading different types  
printf("Enter age: ");  
scanf("%d", &age); // & is required for non-arrays  
  
printf("Enter height: ");  
scanf("%f", &height);  
  
printf("Enter grade: ");  
scanf(" %c", &grade); // Space before %c skips whitespace  
  
printf("Enter name: ");  
scanf("%49s", name); // No & for arrays, limit to 49 chars
```

Always use & for variables (except arrays)!

scanf(): Return Value

```
int a, b;
int result;

printf("Enter two numbers: ");
result = scanf("%d %d", &a, &b);

if (result == 2) {
    printf("Sum: %d\n", a + b);
} else {
    printf("Invalid input! Only read %d value(s)\n", result);
}
```

scanf() returns:

- Number of items successfully read
- EOF (-1) on failure

File I/O



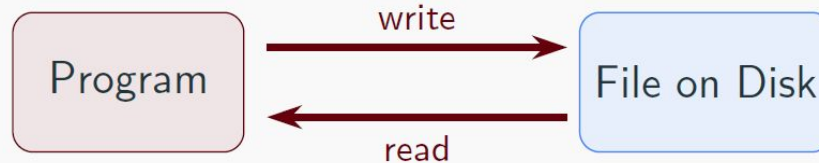
Why File I/O?

Console I/O limitations:

- Data is lost when program ends
- Can't process large datasets
- No way to save results

File I/O enables:

- **Persistence:** Save data permanently
- **Large data:** Process files of any size
- **Sharing:** Exchange data between programs



Opening a File: fopen()

```
FILE *fp; // File pointer

// Open file for reading
fp = fopen("data.txt ", "r");

// Always check if file opened successfully
if (fp == NULL) {
    printf("Error: Could not open file!\n");
    return 1;
}

// ... work with file ...

fclose(fp); // Always close the file!
```

File Modes

Mode	Description	If file doesn't exist
"r"	Read only	Error
"w"	Write only (overwrites)	Creates new file
"a"	Append (write at end)	Creates new file
"r+"	Read and write	Error
"w+"	Read and write (overwrites)	Creates new file
"a+"	Read and append	Creates new file

Important

"w" mode will delete existing file contents!
Use "a" to add to existing file.

Writing to Files

```
FILE *fp = fopen("output.txt", "w");  
if (fp == NULL) return 1;  
  
// Method 1: fprintf() - like printf, but to file  
fprintf (fp, "Name: %s\n", "Alice");  
fprintf (fp, "Score: %d\n", 95) ;  
  
// Method 2: fputs() - write a string  
fputs (" This is a line \n", fp);  
  
// Method 3: fputc() - write a character  
fputc ('X', fp);  
  
fclose (fp);
```


Reading from Files

```
FILE *fp = fopen("input.txt ", "r");
if (fp == NULL) return 1;

char line[100];
int num;

// Method 1: fscanf() - like scanf, but from file
fscanf (fp , "%d", & num );

// Method 2: fgets() - read a line (safe)
fgets (line , 100 , fp);

// Method 3: fgetc() - read a character
char c = fgetc (fp);

fclose (fp);
```

Reading from Files

```
FILE *fp = fopen("numbers.txt", "r");
if (fp == NULL) return 1;

int num, sum = 0, count = 0;

// Read until end of file
while(fscanf(fp , "%d", &num) == 1) {
    sum += num;
    count++;
}

printf("Read %d numbers, sum = %d\n", count, sum);

fclose(fp);
```

Alternative: Use `feof(fp)` to check for end of file.

Complete Example: Student Records

```
typedef struct {
    char name[50];
    int roll;
    float cgpa;
} Student;
// Write students to file
void saveStudents(Student arr[], int n, const char* filename) {
    FILE *fp = fopen(filename, "w");
    if (fp == NULL) return;
    for (int i = 0; i < n; i++) {
        fprintf(fp, "%s %d %.2f\n", arr[i].name, arr[i].roll, arr[i].cgpa);
    }
    fclose (fp);
}
// Read students from file
int loadStudents(Student arr[], const char* filename) {
    FILE *fp = fopen(filename, "r");
    if (fp == NULL) return 0;
    int count = 0;
    while(fscanf(fp, "%s %d %f", arr[count].name, &arr[count].roll, &arr[count].cgpa) == 3) {
        count++;
    }
    fclose(fp);
    return count ;
}
```

Binary vs Text Mode

Mode	Suffix	Use Case
"rb"	Binary read	Images, executables
"wb"	Binary write	Save raw data
"ab"	Binary append	Add binary data
"rb+"	Binary read/write	Modify binary files
"wb+"	Binary read/write (new)	Create binary files

Text mode: Characters may be translated (e.g., $\backslash n \leftrightarrow \backslash r\backslash n$ on Windows)

Binary mode: Exact bytes are read/written as-is

Binary I/O: fread() and fwrite()

```
// Write an array to binary file
int arr [5] = {10 , 20, 30, 40, 50};
FILE *fp = fopen("data.bin", "wb");

// fwrite(pointer, size_of_element, num_elements, file)
fwrite(arr, sizeof(int), 5, fp);
fclose(fp);

// Read array from binary file
int loaded[5];
fp = fopen("data.bin", "rb");

// fread(pointer, size_of_element, num_elements, file)
fread(loaded, sizeof(int), 5, fp);
fclose(fp);
```

Saving Structures to Binary Files

```
typedef struct {
    char name[50];
    int roll;
    float cgpa;
} Student;

// Write structure to binary file
Student s = {"Alice", 101, 3.85};
FILE *fp = fopen("student.dat", "wb");
fwrite(&s, sizeof(Student), 1, fp);
fclose(fp);

// Read structure from binary file
Student loaded;
fp = fopen("student.dat", "rb");
fread(&loaded, sizeof(Student), 1, fp);
fclose(fp);
```

Saving Structures to Binary Files

Move the file position indicator to a specific location:

```
FILE *fp = fopen("data.txt", "r");

// fseek (file , offset , origin )
fseek(fp, 0, SEEK_SET); // Go to beginning
fseek(fp, 0, SEEK_END); // Go to end
fseek(fp, 10, SEEK_SET); // Go to byte 10 from start
fseek(fp, -5, SEEK_END); // Go 5 bytes before end
fseek(fp, 20, SEEK_CUR); // Move 20 bytes forward from current

fclose(fp);
```

Origin constants:

- SEEK_SET: Beginning of file (0)
- SEEK_CUR: Current position (1)
- SEEK_END: End of file (2)

File Positioning: ftell() and rewind()

```
FILE *fp = fopen("data.txt", "r");

// ftell() - Get current position
long pos = ftell (fp);
printf("Current position: %ld\n", pos);

// Read some data ...
fgets(buffer, 100 , fp);

// ftell() again
pos = ftell(fp);
printf("After reading: %ld\n", pos);

// rewind() - Go back to beginning (same as fseek(fp, 0, SEEK_SET))
rewind(fp);

// Now we're at the start again
pos = ftell(fp); // pos is 0

fclose(fp);
```


Getting File Size

```
FILE *fp = fopen("data.txt", "rb");
if (fp == NULL) return -1;

// Go to end of file
fseek(fp , 0, SEEK_END);

// Get position (= file size in bytes)
long size = ftell(fp);

// Go back to beginning
rewind(fp);

printf("File size: %ld bytes\n", size);
fclose(fp);
```

Random Access in Binary Files

```
#define NUM_STUDENTS 100
Student students[NUM_STUDENTS];

// Save all students to binary file
FILE *fp = fopen("students.dat", "wb");
fwrite(students, sizeof(Student), NUM_STUDENTS, fp);
fclose(fp);

// Read ONLY the 50th student (without reading others)
Student s50;
fp = fopen("students.dat", "rb");

// Jump to position of 50th student (index 49)
fseek(fp , 49 * sizeof(Student), SEEK_SET);

// Read just that one student
fread(&s50, sizeof(Student), 1, fp);
fclose(fp);

printf("Student 50: %s\n", s50.name);
```

Error Handling: `ferror()` and `clearerr()`

```
FILE *fp = fopen("data.txt", "r");
if (fp == NULL) {
    perror("Error opening file"); // Print error message
    return 1;
}

char c = fgetc(fp);

// Check for errors
if (ferror(fp)) {
    printf("Error reading from file!\n");
    clearerr(fp); // Clear error flag
}

// Check for end of file
if (feof(fp)) {
    printf("Reached end of file\n");
}

fclose(fp);
```

Buffering: fflush()

```
FILE *fp = fopen("log.txt", "w");

fprintf(fp, "Starting process...\n");
fflush(fp); // Force write to disk NOW (don't wait for buffer)

// Long running process ...
fprintf(fp, "Step 1 complete\n");
fflush(fp);

fprintf(fp, "Step 2 complete\n");
fflush(fp);

fclose(fp); // fclose() also flushes automatically
```

Why use fflush()?

- Ensure data is written before crash
- Real-time logging
- Synchronize output from multiple processes

String I/O: sprintf() and sscanf()

```
char buffer[100];
int age = 25;
float height = 5.9;

// sprintf() - Write formatted string to buffer (not file)
sprintf(buffer, "Age: %d, Height: %.1f", age, height);
printf("Buffer: %s\n", buffer); // Age: 25, Height: 5.9

// sscanf() - Read from string (not stdin)
char data[] = "Alice 101 3.85";
char name[50];
int roll;
float cgpa;

sscanf(data, "%s %d %f", name, &roll, &cgpa);
printf("Name: %s, Roll: %d, CGPA: %.2f\n", name, roll, cgpa);
```

sprintf/sscanf work on strings, not files!

Structure Pointers and Arrow Operator

```
typedef struct {  
    char name[50];  
    int roll;  
} Student;  
  
Student s = {"Alice", 101};  
Student *ptr = &s;  
  
// Using pointer: two ways to access members  
printf("Name : %s\n", (*ptr).name); // Dereference then dot  
printf("Roll: %d\n", ptr->roll);  
  
// Modifying through pointer  
ptr->roll = 102;  
strcpy(ptr->name, "Bob");
```

`ptr->member` is shorthand for `(*ptr).member`

printf(): Format Specifiers

Function	Purpose
fopen()	Open a file
fclose()	Close a file
fgetc(), fputc()	Read/write single character
fgets(), fputs()	Read/write string
fscanf(), fprintf()	Formatted read/write
fread(), fwrite()	Binary read/write
fseek()	Move file position
ftell()	Get current position
rewind()	Go to beginning
feof()	Check end of file
ferror()	Check for errors
clearerr()	Clear error flags
fflush()	Flush output buffer
remove()	Delete a file
rename()	Rename a file

Summary



Summary

Console I/O:

- `getchar()/putchar()` for chars
- `fgets()/puts()` for strings

Formatted I/O:

- `printf()` format specifiers
- `scanf()` with `&` operator
- Width and precision control

File I/O:

- `fopen()/fclose()`
- Modes: `r`, `w`, `a`, `r+`, `w+`, `a+`
- `fprintf()/fscanf()`
- Always check for errors!

Acknowledgement

- Mohammad Sadat Hossain, Lecturer, CSE, BUET